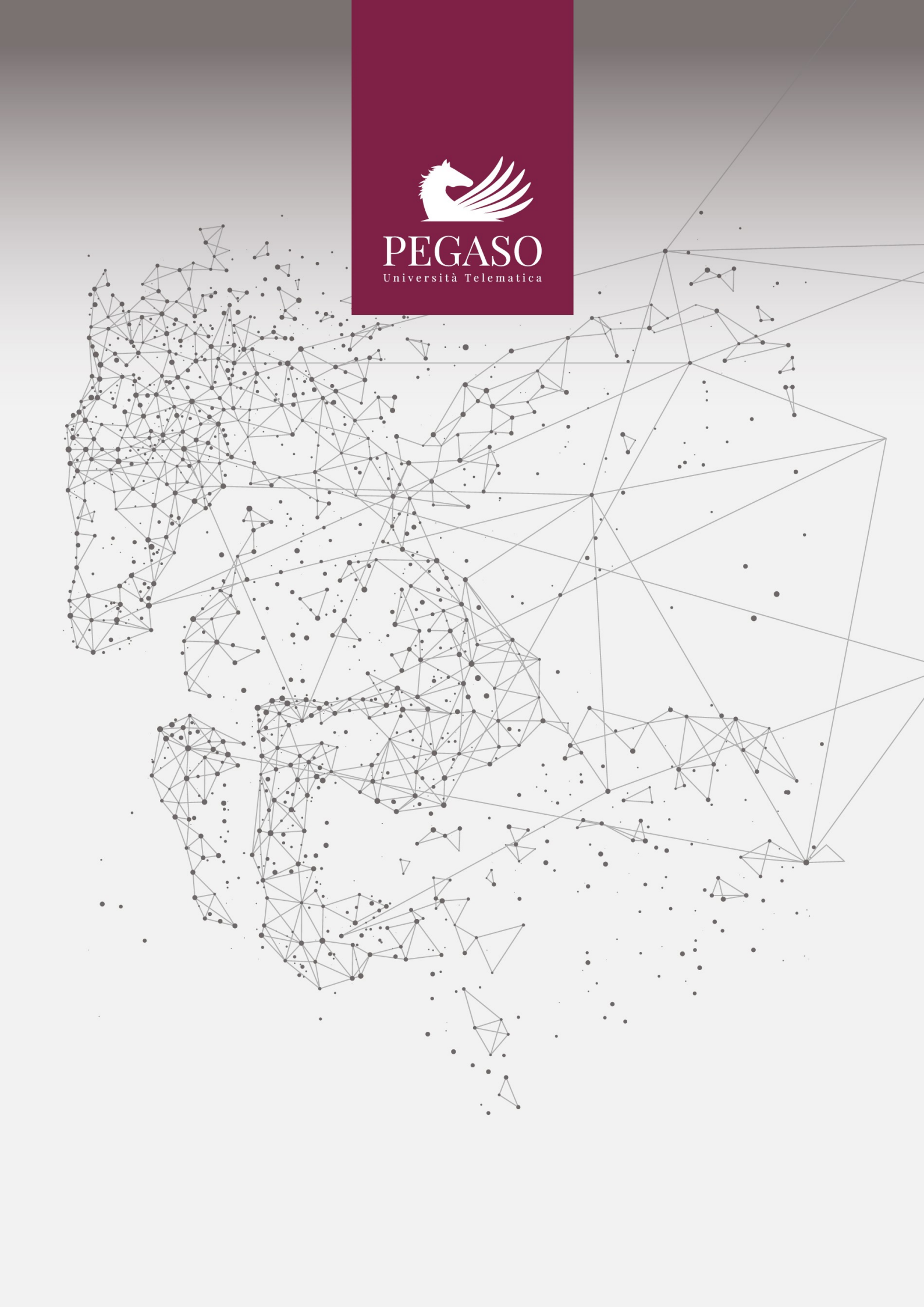

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. PRESENTAZIONE DEL PROBLEMA E SOLUZIONE "NAIVE" 3

2. SOLUZIONE EFFICIENTE 5

3. INTRODUZIONE AL CONCETTO DI COMPLESSITÀ.....8

BIBLIOGRAFIA 11

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Presentazione del problema e soluzione "naive"

Analizziamo il problema della ricerca di un elemento all'interno di un array. Supponiamo di avere un array di interi su cui non facciamo ipotesi "aggiuntive" (grandezza array, distribuzione degli elementi ecc.).

Per semplicità ammettiamo che l'array abbia 10 elementi:

[0,1,2,3,4,5,6,7,8,9]

Supponiamo ora di voler ricercare un elemento all'interno dell'array e vogliamo che venga restituito il valore dell'indice dell'elemento (se presente) oppure -1 (se assente).

Una prima soluzione al problema consiste nel confrontare ogni singolo elemento dell'array con l'elemento da ricercare. Se ad esempio l'elemento da ricercare è lo 0 ci troviamo in una situazione fortunata dal momento che lo 0 è il primo elemento dell'array, pertanto potremo restituire direttamente 0 (l'indice dell'elemento).

Se l'elemento da ricercare è il 4 ci troviamo in una situazione "media" dal momento che il 5 è "quasi" al centro dell'array, pertanto dobbiamo scorrerne solo una metà e poi restituire 5 (l'indice dell'elemento).

Se l'elemento da ricercare è il 9 ci troviamo in una situazione sfortunata dal momento che il 9 è l'ultimo elemento dell'array, pertanto dovremo restituire 9 (l'indice dell'elemento).

Se l'elemento da ricercare non è presente nella lista ci troviamo ancora una volta in una situazione sfortunata dal momento che siamo comunque costretti a scorrere tutto l'array per poi restituire -1 (elemento non presente).

Abbiamo dunque:

- Caso fortunato: 1 solo confronto.
- Caso medio: 6 confronti.
- Caso sfortunato: 10 confronti.
- Elemento non presente: 10 confronti.

Possiamo dunque concludere dicendo che se non facciamo supposizioni particolari sulla struttura dell'array, il numero dei confronti che andiamo a fare per poter restituire l'indice dell'elemento (se presente) è proporzionale alla lunghezza dell'array stesso: se siamo fortunati ci basterà un confronto, se siamo sfortunati dovremo compiere "al più" n confronti (essendo n la lunghezza dell'array stesso).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Tale soluzione proposta è stata indicata come "naive" poiché è una soluzione semplice ed ingenua, che richiede poco effort da un punto di vista di scrittura del codice ed analisi del problema e che ci garantisce comunque una soluzione al problema.

Una possibile implementazione in c++ dell'algoritmo è la seguente:

```
int ricercaSequenziale(int array[],int len,int value) {
  int i=0;
  while (i<len) {
    if (array[i]==value)
      return i;
    i++;
  }
  return -1;
}
```

Figura 1 *- Ricerca sequenziale*

La funzione ha come parametri in ingresso l'array stesso, la lunghezza dell'array (non vogliamo usare librerie o funzioni che ci forniscano tale valore) ed il valore da ricercare.

Da notare che l'array utilizzato nell'esempio precedente era "particolare" in quanto "ordinato" in maniera crescente; tuttavia, se anche non lo fosse stato (se anche avesse avuto valori totalmente randomici), l'esito dell'analisi sarebbe stato identico.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Soluzione efficiente

Analizziamo ora una "variante" al problema presentato in precedenza: supponiamo di dover ricercare come prima un elemento all'interno dell'array e vogliamo che venga restituito il valore dell'indice dell'elemento (se presente) oppure -1 (se assente) ma imponiamo come condizione che l'array sia "ordinato".

Ci troviamo cioè nello scenario dell'esempio precedente ma stavolta ammettiamo per ipotesi che l'array ci venga fornito già ordinato in maniera crescente.

Data questa circostanza possiamo permetterci di lavorare con un algoritmo differente, ragionando nella seguente maniera:

[0,1,2,3,4,5,6,7,8,9]

Supponiamo che l'elemento da ricercare sia il 7.

Se l'array è ordinato posso ragionare nella seguente maniera:

- Considero l'elemento medio dell'array.
- Verifico se l'elemento medio dell'array è quello ricercato.
- Se lo è, restituisco direttamente l'indice dell'elemento medio.
- Se non lo è opero nella seguente maniera:
    - Se l'elemento ricercato è minore dell'elemento nella posizione media, analizzo solamente la prima metà dell'array.
    - Se l'elemento ricercato è maggiore dell'elemento nella posizione media, analizzo solamente la seconda metà dell'array.
- Procedo ricorsivamente finché non trovo l'elemento (ed in tal caso restituisco l'indice) oppure finché non posso più suddividere l'array (ed in tal caso restituisco -1).

Sto dunque utilizzando un approccio "ricorsivo" al problema e posso usufruire di questo approccio dal momento che ho il vincolo di array "ordinato" in maniera crescente.

Se dunque l'elemento che voglio cercare è l'8 lavoro gli steps da seguire saranno:

- Considero l'elemento "medio", cioè nella posizione 5 → array[5]=5
- Tale elemento è quello che cercavo? La risposta è no
- Tale valore è minore o maggiore di quello che cercavo? È minore, pertanto vado ad analizzare la seconda parte dell'array quella che parte dall'indice 6 al 9 → [6,7,8,9]
- Considero l'elemento "medio", cioè nella posizione 8 → array[8]=8

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Tale elemento è quello che cercavo? La risposta è sì → restituisco l'indice 8

È facile provare con altri elementi e ci si rende conto che anche nel caso sfortunato di elemento non presente nell'array, non sono costretto ad eseguire tutti i confronti con tutti gli elementi, ma con un numero ridotto di elementi, più precisamente pari ad un log2.

Rispetto alla soluzione precedente in cui ero costretto a fare n confronti, qui mi trovo a doverne fare $\log_2 n$ e quindi ho certamente un vantaggio dal punto di vista computazione e dunque di prestazioni.

Facciamo qualche calcolo:

- Se ad esempio n fosse pari a 10^8 e mi trovassi su una macchina in grado di eseguire 10^5 confronti al secondo, il tempo di calcolo nel caso peggiore della soluzione naive sarebbe: $10^8 / 10^5 = 10^3$ → circa 16 minuti.
- Nel secondo caso invece il calcolo sarebbe $\log_2(10^8) / 10^5 = 0{,}0002657$ → circa 4 minuti.

Il tempo si è notevolmente ridotto! È vero che ci siamo messi in una condizione differente (ipotesi di ordinamento), ma è anche vero che se avessimo continuato ad utilizzare l'algoritmo naive, le prestazioni sarebbero state ben differenti.

Di seguito il codice dell'algoritmo:

```
int ricercaBinariaSplit(int array[],int value,int sinistra,int destra) {
  if (destra < sinistra)
    return -1;
  else {
    int medio=(sinistra+destra)/2;
    if (array[medio]==value)
      return medio;
    if (medio==0)
      return -1;
    else if (value<array[medio])
      return ricercaBinariaSplit(array,value,sinistra,medio-1);
    else
      return ricercaBinariaSplit(array,value,medio+1,destra);
  }
}

int ricercaBinaria(int array[],int len,int value) {
  return ricercaBinariaSplit(array,value,0,len);
}
```

Figura 2 *- Ricerca binaria*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Come si evince dal codice, si è utilizzata una funzione ricercaBinaria generale (con gli stessi parametri in ingresso della funzione di ricercaSequenziale), e poi si è utilizzata una funzione ricercaBinariaSplit che opera sul sottoinsieme dell'array in maniera ricorsiva.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Introduzione al concetto di complessità

L'esempio appena riportato mette in luce degli aspetti interessanti su cui vale la pena soffermarsi:

- Dato un problema possono esistere più algoritmi che lo risolvono.
- I vincoli del problema sono importanti e vanno analizzati opportunamente, al fine di individuare la soluzione migliore.
- Un cambiamento dell'algoritmo può avere impatto significativo sulle performance.
- Entrambi gli algoritmi analizzati sono corretti sintatticamente e semanticamente ma differiscono in termini di pragmatica: il secondo è più efficiente del primo.

Quanto analizziamo un algoritmo, dobbiamo fare analizzarne i seguenti aspetti:

- Correttezza.
- Completezza.
- Complessità.

Un algoritmo è corretto quando restituisce sempre una risposta corretta: la parola chiave in questa frase è "sempre"; non è dunque ammissibile che esistano delle istanze per cui l'algoritmo restituisce una risposta errata: se cioè avvenisse significherebbe che l'algoritmo non è corretto.

Un algoritmo è completo quando ogni risposta che bisognerebbe restituire è effettivamente restituita (ogni risposta corretta prima o poi è effettivamente restituita). Tipicamente un problema di completezza di potrebbe avere quando sono ammessi più output: in tal caso l'algoritmo è completo se li restituisce tutti e non soltanto una parte. Nella maggioranza delle analisi che facciamo, ammettiamo una sola soluzione al problema; pertanto, la completezza va a coincidere con la correttezza e con la terminazione dell'algoritmo stesso. In altre parole: quando il problema è "funzionale" (ci si aspetta cioè una sola soluzione) allora correttezza e completezza vanno a coincidere.

Una volta che ci siamo accertati del fatto che il nostro algoritmo è corretto, possiamo dunque analizzarne la complessità.

Fornire la complessità dell'algoritmo corretto, significa dare "almeno" un tetto alla complessità del problema risolto dall'algoritmo; se l'algoritmo non è il migliore per risolvere quel problema, allora quella complessità non sarà ottima e noi non avremmo risposto alla domanda: "qual è la complessità del mio problema?"; ma avremmo risposto solo la domanda "qual è la complessità del mio algoritmo?".

Analizziamo un altro esempio ed applichiamo i due algoritmi cercando di capire che tipo di impatto hanno in termini di complessità.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Definizione del problema:

Ho un insieme di monete (n monete) tutte identiche tra loro, tranne una che è falsa. La loro differenza sta nel peso: la moneta falsa ha un peso maggiore rispetto alla moneta vera. Abbiamo a disposizione una funzione weight(x, y) che restituisce:

- *x: se il peso complessivo di x è maggiore del peso complessivo di y.*
- *y: se il peso complessivo di y è maggiore del peso complessivo di x.*
- *0: se il peso complessivo di x è identico al peso complessivo di y.*

Dobbiamo costruire un algoritmo per riuscire a capire qual è la moneta falsa cercando di ridurre al minimo le chiamate alla funzione weight.

Adottiamo 2 strategie differenti, analoghe a quanto descritto in precedenza:

- **Strategia1**: "naive".
- **Strategia2**: "intelligent".

Nel caso di strategia "naive" il procedimento è il seguente:

1. Se l'insieme è formato da 2 sole monete, eseguo il confronto tra le 2:
    a. Se il peso della prima è maggiore, restituisco la moneta in esame come "falsa".
    b. Se il peso della seconda è maggiore, restituisco la moneta in esame come "falsa".
2. Se l'insieme è formato da più monete, seleziono la prima moneta dell'insieme e la confronto con tutte le altre:
    a. Se il peso è maggiore: restituisco la moneta in esame come "falsa".
    b. Se il peso è minore: restituisco la moneta selezionata dall'insieme come falsa.
    c. Se il peso è identico proseguo al passo 2.
3. Considero l'insieme delle monete rimanenti e procedo con lo step 1 confrontando la moneta con le rimanenti dell'insieme

Ovviamente tale procedimento prevede un numero "importante" di confronti; se le monete sono n e ci troviamo nel caso "sfortunato" abbiamo:

- n-1 confronti (seleziono la prima moneta);
- n-2 confronti (seleziono la seconda moneta);
- […]
- n-n+1 confronti (seleziono la penultima moneta).

In totale abbiamo eseguito dunque (nel caso sfortunato):

$$\sum_{k=1}^{n-1} n-k$$

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel caso di strategia "intelligent" il procedimento è il seguente:

1. Se l'insieme delle monete è "pari", divido a metà l'insieme di monete e procedo come nel punto 3.
2. Se l'insieme delle monte è dispari, isolo l'elemento a metà; divido a metà l'insieme di monete rimanenti e procedo nel punto 3.
3. Confronto il peso dei due insiemi:
    a. Se il numero di elementi è pari a 2:
        i. Se il peso del primo elemento è maggiore del secondo, il primo elemento è il falso.
        ii. Se il peso del secondo elemento è maggiore del primo, il secondo elemento è il falso.
        iii. Se il peso dei due elementi è identico vuol dire che l'elemento che avevo isolato (essendo l'insieme dispari) è quello falso.
    b. Se il numero di elemento è maggiore di 2:
        i. Se il peso del primo insieme è maggiore eseguo la procedura su questo insieme.
        ii. Se il peso del secondo insieme è maggiore eseguo la procedura su questo insieme.
        iii. Se il peso dei due insiemi è identico vuol dire che l'elemento che avevo isolato (essendo l'insieme dispari) è quello falso.

In totale abbiamo eseguito dunque (nel caso sfortunato) il seguente numero di confronti: $\log_2(n)$.

In questo scenario, a parità di condizioni iniziali possiamo verificare come un algoritmo differente comporta un sostanziale cambio di complessità (e di conseguenza di prestazioni).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Alan Bertossi, Alberto Motresor: Algoritmi e strutture di dati, Città Studi Edizioni, terza edizione.
- C. Demetrescu, I. Finocchi, G. F. Italiano: Algoritmi e strutture dati, McGraw-Hill, seconda edizione.
- Crescenzi, Gambosi, Grossi: Strutture di Dati e Algoritmi, Pearson/Addison-Wesley.
- Sedgewick: Algoritmi in C, Pearson, 2015.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).